MongoDB Student Hackathon - Challenge Tracks

Choose one track and build a working prototype.

Each track is open-ended. You can use public data, synthetic data, team-created material, third-party APIs, or data generated by your own product. Keep the input set small, make setup repeatable, and focus on completing one user workflow end to end.

Coding agents such as Codex or Claude Code are allowed. Be ready to explain your architecture, data model, and the choices you made.

Before the event

If you're new to building AI applications, we recommend AI Data Strategy with MongoDB as an introduction before the event.

Each track also includes relevant MongoDB Skill Badges for preparation or further learning afterwards. These are optional resources, and completing them is not required to participate.

Using Vector Search? You'll need embeddings, which represent your content as numerical vectors, and a Vector Search index configured to search them. MongoDB's automated embedding can simplify setup where supported. Check the linked guide for requirements. If you generate embeddings yourself, configure the index to match your model's output and use the same embedding model for your content and queries.

Track 1: Agentic Memory

Build an application or agent that remembers useful information and changes how it behaves over time.

Challenge Brief

Build an application or agent with persistent memory across interactions or sessions. Decide what should be remembered, how it is retrieved, and when it should be updated, corrected, or forgotten. Represent memory that matters - facts, preferences, decisions, outcomes, constraints, feedback, or learned strategies. The stored memory must visibly change the system's next response, recommendation, plan, or action. The domain and interface are up to you.

Example Demo Flow

- Start with an interaction that creates a useful memory.
- Open a later session and show the retrieved memory changing the result or action.
- Inspect the stored memory and explain why it was selected.
- Demonstrate one difficult case: a correction, expiry, deletion, conflict, or private information.

Using MongoDB

- Model each memory as a document with an owner, type, content, source, timestamps, and lifecycle fields.
- Use MongoDB Vector Search for semantic recall, with filters to scope results by user, workspace, memory type or recency. See the setup note above for embedding and index requirements.
- Use a TTL index on a date field for time-based memory expiry. Handle other rules, such as deleting a memory when a flag changes, in application logic.
- Use change streams when multiple users or agents need live updates.

Relevant Docs

- Build AI Agents with MongoDB - Agent architecture, retrieval, and persistent memory.
- Data Modeling - Structure memory documents and their relationships.
- MongoDB Vector Search - Recall relevant memories with semantic search and metadata filters.
- TTL Indexes - Automatically expire time-limited memories.
- Change Streams - React to memory updates across users or agents.

Relevant MongoDB Skill Badges

- Memory for AI Applications
- Relational to Document Model
- Schema Design Optimization
- Vector Search Fundamentals

Example Project Ideas

- **Shared Context for Teammates' Agents** - separate coding agents share decisions, task ownership, file changes, constraints, and open questions.
- **Adaptive Incident Response Agent** - an agent remembers previous incidents, attempted fixes, and outcomes while detecting when old information no longer applies.
- **Long-Running Decision Assistant** - an assistant remembers assumptions, decisions, and results across a project and surfaces them when relevant.

Track 2: RAG Engine

Build a product that turns your own source material into grounded, useful results.

Challenge Brief

Build a RAG application around a problem of your choosing. Source a focused collection of documents, webpages, images, transcripts, or structured records - from a public source, your own material, or a set you generate with an LLM. Retrieve evidence from more than one source or section, then use it to answer, compare, recommend, investigate, or generate an artefact. Show the source behind every important result, and handle at least one difficult or unanswerable request clearly.

Example Demo Flow

- Show how a user adds or imports a small collection and how it becomes searchable.
- Run one request that requires evidence from multiple sources or distant sections.
- Open the citations and show the exact passages or source items used.
- Try a weak, conflicting, or unanswerable request and show how the product responds.

Using MongoDB

- Store original sources, chunks, embeddings, metadata, and source identifiers together. Use MongoDB Vector Search for semantic retrieval.
- Add metadata filters for dates, owners, or content types. Use hybrid search when exact names, codes, or keywords also matter.
- Keep a small set of test requests and expected source IDs so you can compare retrieval changes as you build.

Relevant Docs

- RAG with MongoDB - Build the ingestion, retrieval, and generation workflow.
- MongoDB Vector Search - Index embeddings and filter retrieval by source metadata.
- Hybrid Search Tutorial - Combine keyword and semantic search results.
- Explain Vector Search Results - Inspect query execution when debugging retrieval.

Relevant MongoDB Skill Badges

- Vector Search Fundamentals
- Vector Search Performance
- RAG with MongoDB
- Retrieval Evaluation

Example Project Ideas

- **Technical Due Diligence Assistant** - combine architecture documents, incident reports, and a repository summary into an evidence-backed engineering risk brief.
- **Policy Change Tracker** - compare document versions, identify changed obligations, and link each finding to the relevant passage.
- **Multimodal Repair Assistant** - combine manuals, error logs, and an inspection image to suggest next steps with cited instructions.

Track 3: Developer Tools for Production use cases

Build a tool that helps another developer understand, debug, or safely change a software or AI project.

Challenge Brief

Build a developer tool that improves one part of the build, test, debug, or release workflow. Pick one precise question or action - why a run failed, what changed, whether a release is safe, or which consumers will break. The tool can be a CLI, SDK, API, webhook, GitHub integration, editor extension, or a focused web interface. Run it against your own project or a generated sample, store the relevant history in MongoDB, and turn that history into a diagnosis, comparison, warning, or action. An LLM is optional.

Example Demo Flow

- Start by running the tool or triggering its integration against a real or generated project.
- Show the project activity or history captured in MongoDB.
- Run a second scenario with a meaningful change and show the result updating.
- Trigger one practical failure case: a duplicate event, incomplete input, failed job, or invalid configuration.

Using MongoDB

- Store runs and events with stable IDs, timestamps, and links to the project or version.
- Create indexes for the questions your tool asks most often, then use aggregation pipelines to compare runs or derive findings.
- Use schema validation for required event fields while keeping source-specific payloads flexible.
- Use change streams for live views or notifications, and make repeated ingestion safe with stable event IDs.

Relevant Docs

- Data Modeling - Design collections for runs, events, and project versions.
- Aggregation Operations - Compare runs and summarize event history.
- Schema Validation - Validate required fields in incoming events.
- Unique Indexes - Enforce unique event IDs to prevent duplicate records.
- Change Streams - Update live views when new events arrive.

Relevant MongoDB Skill Badges

- Relational to Document Model
- Fundamentals of Data Transformation
- Advanced Schema Patterns and Antipatterns

Example Project Ideas

- **Agent Run Debugger** - capture prompts, retrieved context, tool calls, outputs, and errors, then compare a successful run with a failed run.
- **API Change Impact Mapper** - compare two API specifications with a small consumer map, identify breaking changes, and generate migration tasks.
- **Coding Agent Change Verifier** - connect a task, agent plan, code diff, and test results, then flag requirements without evidence and risky changes without coverage.

General Hackathon Advice

- **Find your spark.** For inspiration, check out examples like Rajan Agarwal's projects, then put your own spin on an idea that excites you.
- **Try something new.** Build with MongoDB and use the day to explore an unfamiliar framework, API, or interface. A small working experiment is a great place to start.
- **See what models can really do.** Try a task you assumed was out of reach, test where the model succeeds or struggles, find its limits and build around what you discover.
- **Talk to the room.** Share what you're building, ask for help, and meet people whose skills complement yours. You might find a teammate or unlock a new direction.
- **Have fun with it.** Make something you're curious about, check out other teams' demos, and leave with new friends and something you learned.